Chapter 4: Distributed Database System

Instructor: Melaku M.

Target Group: G3 SE

Outline

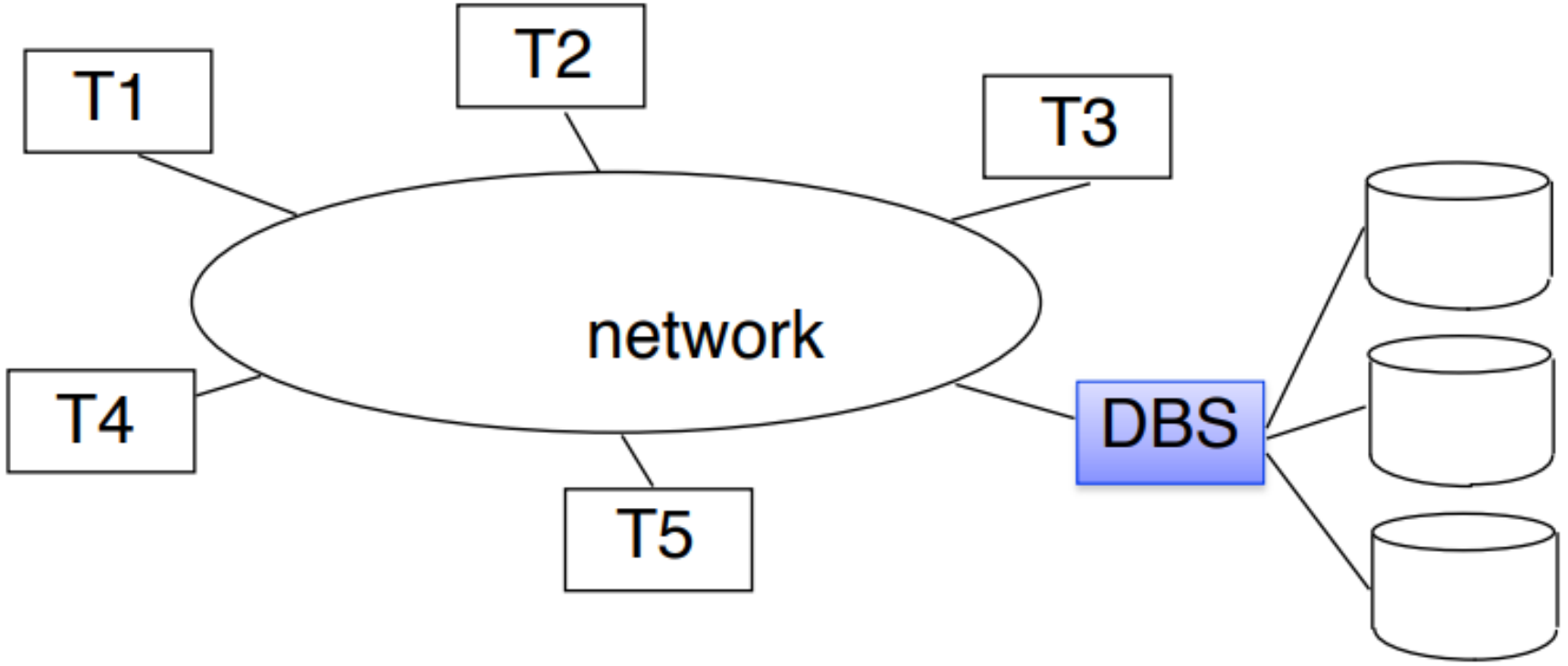
- Concepts of distributed database
- Distributed database design
- Distributed query processing
- Distributed transaction management and recovery

Concepts of distributed database

- **Distributed database** is a collection of **interrelated data** that is distributed across multiple physical locations. These locations may be on different computers, servers, or even in different geographical regions.
- **Logically integrated** and **appears as a single database to the user**, even though it is physically distributed across multiple sites.
- Database systems that run on each site are independent of each other.
- Transactions may access data at one or more sites.

Centralized DBS

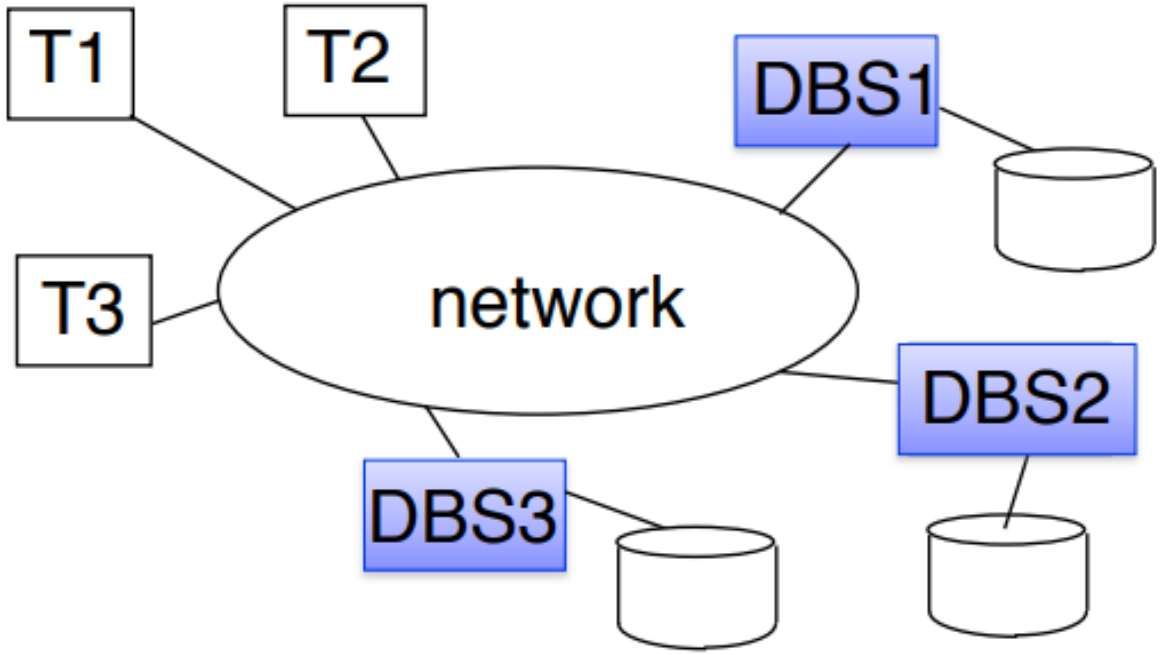
- Logically integrated, Physically centralized



Traditionally: one large mainframe DBMS + n “stupid” terminals

Distributed DBS

- Data logically integrated (i.e., access based on one schema).
- Data physically distributed among multiple database nodes.
- Processing is distributed among multiple database nodes.



Why a Distributed DBS?

Performance via parallel execution
- more users
- quick response

More data volume
- max disks on multiple nodes

Traditionally: m mainframes for the DBMSs + n terminals

Advantages of Distributed DBMS

- **Fault Tolerance**
  - The system continues to operate even if some nodes fail.
  - Achieved through data replication and redundancy.
  - Redundancy ensures availability during failures.
- **Scalability**
  - Distributed databases can handle a growing amount of data and traffic by adding more nodes.
- **Improved Performance**: Localized access to data reduces latency.
- **Geographical Distribution**: Useful for global applications.
- **Transparency of distribution**
- **Efficiency**

Problems/Challenges of Distributed Databases

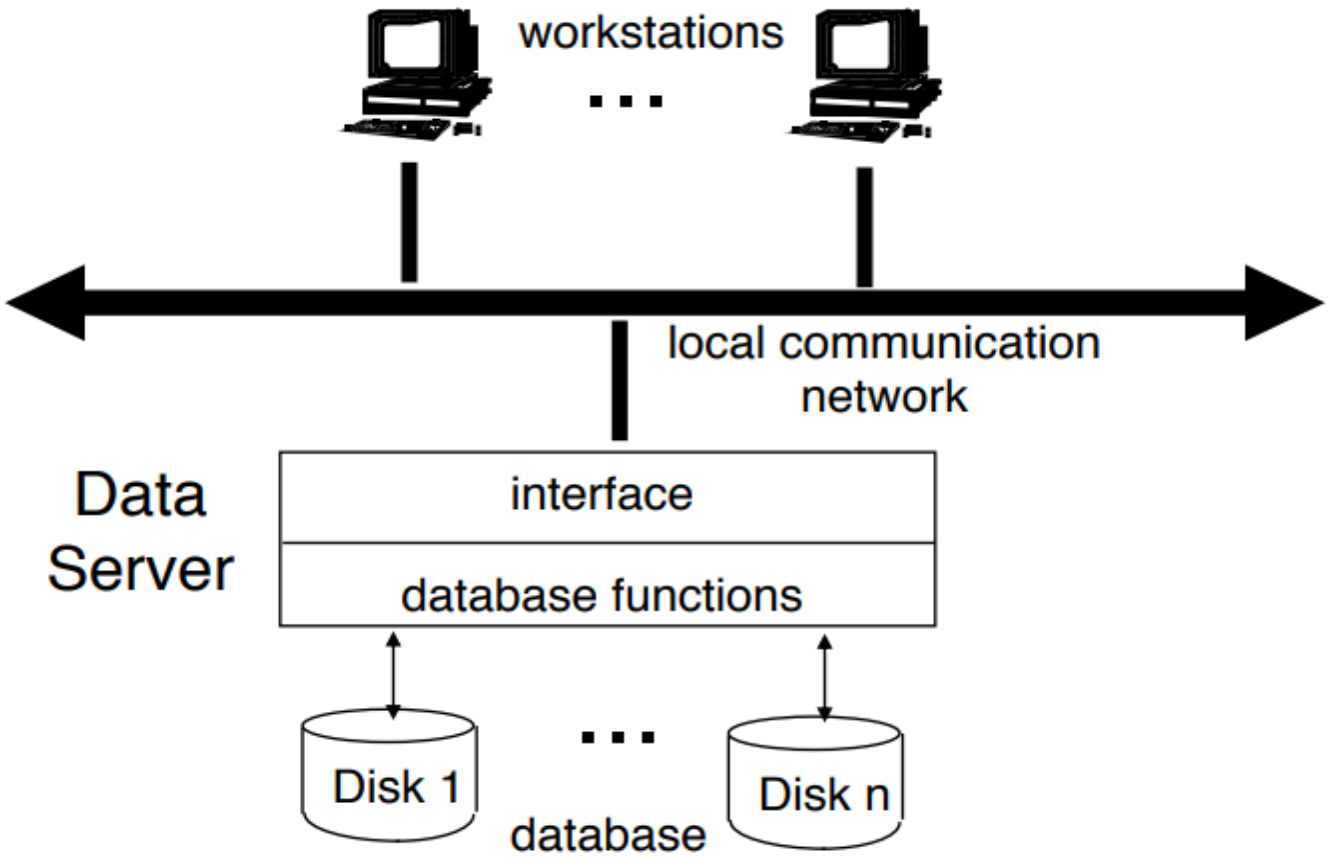
– Complexity of design, implementation and maintain.
– Data consistency: Maintaining data consistency across nodes is difficult.
– Failure recovery
– Latency: Network delays can impact performance.
– Security: More nodes increase the attack surface for potential breaches.

Problems/Challenges of Distributed Databases

- **Need for complex and expensive software** − DDBMS demands complex and often expensive software to provide data transparency and co-ordination across the several sites.
- **Processing overhead** − Even simple operations may require a large number of communications and additional calculations to provide uniformity in data across the sites.
- **Data integrity** − The need for updating data in multiple sites pose problems of data integrity.
- **Overheads for improper data distribution** − Responsiveness of queries is largely dependent upon proper data distribution. Improper data distribution often leads to very slow response to user requests.

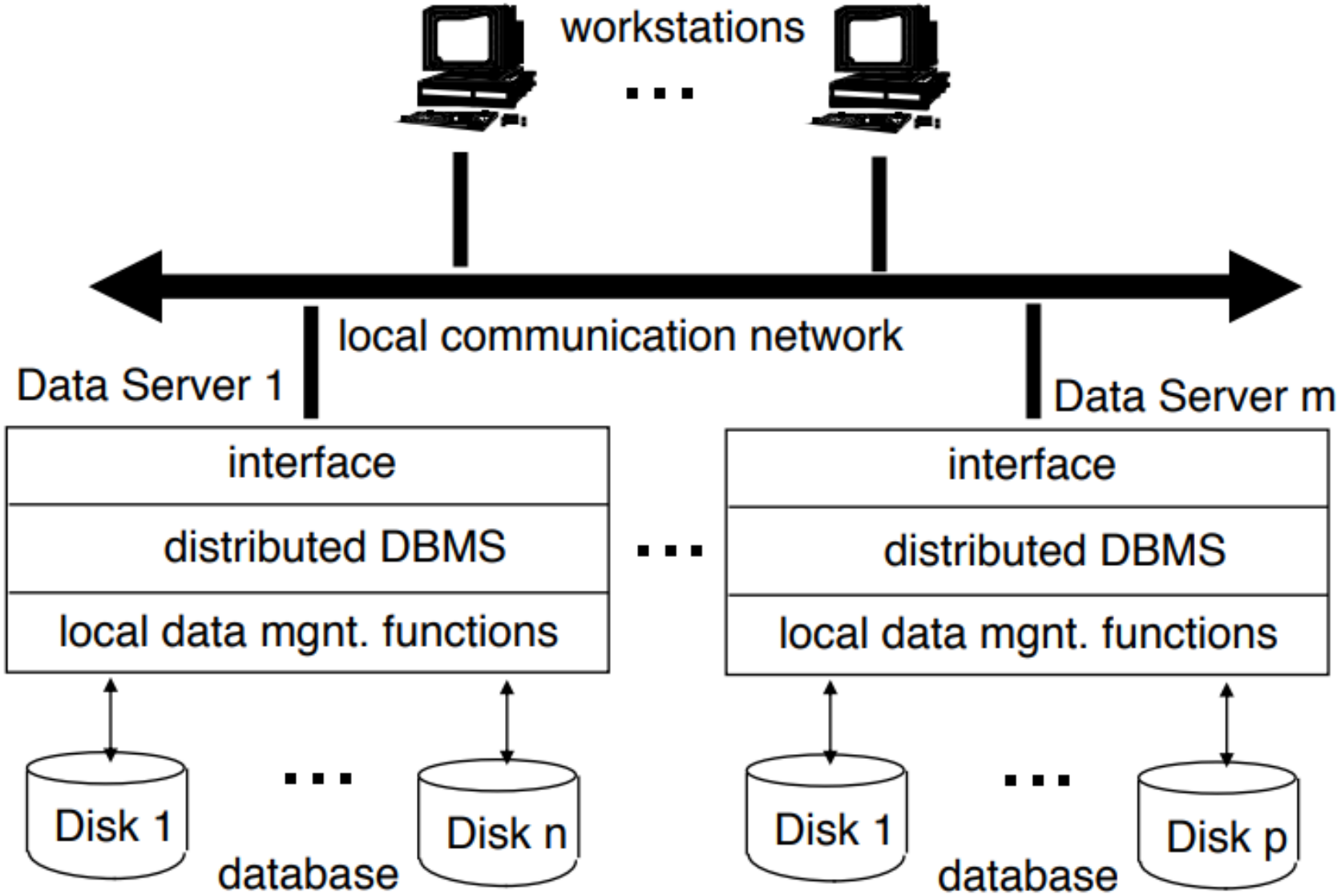
Clients with Centralized Server Architecture

data (object) server + n smart clients (workstations)



- Objects stored and administered on server
- Objects processed (accessed and modified) on workstations [sometimes on the server too]

Clients with Distributed Server Architecture



Types of Distributed Databases

- Distributed databases can be classified into the following categories:

  a. **Homogeneous Distributed Database**

  b. **Heterogeneous Distributed Database**

a. Homogeneous Distributed Databases

- **In a homogeneous distributed database:**
  - All sites have identical software
    - All participating nodes (databases) use the same database management system (DBMS) and schema.
  - Are aware of each other and agree to cooperate in processing user requests.
  - Appears to user as a single system
  - Easier to manage and maintain.
  - Example: Multiple MySQL databases in different locations.

b. Heterogeneous distributed database

- In heterogeneous distributed database:
  - Different sites may use different schemas and software(DBMS).
  - Example: One node using MySQL and another using PostgreSQL
  - Requires middleware or translation layers for communication.
    - Difference in schema is a major problem for query processing.
    - Difference in software is a major problem for transaction processing.
  - Sites may not be aware of each other and may provide only limited facilities for cooperation in transaction processing.

Key Features of Distributed Database

A. Autonomy

- Determines extent to which individual nodes can operate independently.
- **Design autonomy:** Independence of data model usage and transaction management techniques among nodes
- **Communication autonomy:** Determines the extent to which each node can decide on sharing information with other nodes
- **Execution autonomy**

Cont’d

B. Distributed Data Storage

- **Assume relational data model**
  - ❖ **Replication:** System maintains multiple copies of data, stored in different sites, for faster retrieval and fault tolerance.
  - ❖ **Fragmentation:** Relation is partitioned into several fragments stored in distinct sites
  - ❖ **Replication and fragmentation can be combined**
    - ➢ Relation is partitioned into several fragments: system maintains several identical replicas of each such fragment.

C. Data Replication

- Copying data to multiple nodes/sites to improve availability and fault tolerance.
- A relation or fragment of a relation is **replicated** if it is stored redundantly in two or more sites.
- **Partial replication**, only some fragments/relations are replicated on selected nodes. Balances fault tolerance with storage and communication costs.
- **Full replication** of a relation is the case where the relation is stored at all sites. Fully redundant databases are those in which every site contains a copy of the entire database
- Types of replication:
  - ✓Synchronous Replication: Changes are propagated immediately to all replicas.
  - ✓Asynchronous Replication: Changes are propagated with some delay.

Replication(cont.)

❖Advantages of Replication

- ✓**Availability**: failure of site containing relation r does not result in unavailability of r is replicas exist.
- ✓**Parallelism**: queries on r may be processed by several nodes in parallel.
- ✓**Reduced data transfer:** relation r is available locally at each site containing a replica of r.
- ✓**Fault Tolerance**: If one node fails, data is still available on other nodes.
- ✓**Improved Performance**: Queries can access the nearest replica, reducing latency.
- ✓**Load Balancing**: Distributes query load across replicas.

Replication(cont.)

❖Disadvantages of Replication

- ✓**Increased cost of updates:** each replica of relation r must be updated.
- ✓**Increased complexity of concurrency control:** concurrent updates to distinct replicas may lead to inconsistent data unless special concurrency control mechanisms are implemented.
- ✓**Consistency Management**: Ensure all replicas are updated during write operations.
- ✓**Increased Overhead**: Storage and communication costs increase with replication.

D. Data Fragmentation

❖**Fragmentation** is the process of dividing a database into smaller, more manageable pieces (fragments). These fragments can be distributed across nodes to improve performance and availability.

❖**Types of Fragmentation:**

- **Horizontal Fragmentation**
- **Vertical Fragmentation**

D. Data Fragmentation(cont’d)

Horizontal Fragmentation

- ❖Divides a table into subsets of rows based on a condition.
- ❖Each fragment contains a subset of rows, and the union of all fragments reconstructs the original table.
- ❖Example:
  - Employee_US = SELECT * FROM Employee WHERE Country = 'US';
  - Employee_EU = SELECT * FROM Employee WHERE Country = 'EU';

D. Data Fragmentation(cont'd)

Vertical Fragmentation

- Divides a table into subsets of columns.
- Each fragment contains specific columns, and the join of all fragments reconstructs the original table.
- Example:
  - Employee_Personal = SELECT ID, Name, Address FROM Employee;
  - Employee_Job = SELECT ID, Salary, Department FROM Employee;

D. Data Fragmentation(cont’d)

Mixed (Hybrid) Fragmentation

- Combines horizontal and vertical fragmentation.
- **Example**:

  First, apply horizontal fragmentation (split by country), then apply vertical fragmentation (split by columns) to each fragment.

Horizontal Fragmentation of account Relation

branch_name	account_number	balance
Hillside	A-305	500
Hillside	A-226	336
Hillside	A-155	62

$$account_1 = \sigma_{branch_name="Hillside"}(account)$$

branch_name	account_number	balance
Valleyview	A-177	205
Valleyview	A-402	10000
Valleyview	A-408	1123
Valleyview	A-639	750

$$account_2 = \sigma_{branch_name="Valleyview"}(account)$$

Vertical Fragmentation of employee_info Relation

branch_name	customer_name	tuple_id
Hillside	Lowman	1
Hillside	Camp	2
Valleyview	Camp	3
Valleyview	Kahn	4
Hillside	Kahn	5
Valleyview	Kahn	6
Valleyview	Green	7

$deposit_1 = \Pi_{branch_name,\ customer_name,\ tuple_id}(employee_info)$

account_number	balance	tuple_id
A-305	500	1
A-226	336	2
A-177	205	3
A-402	10000	4
A-155	62	5
A-408	1123	6
A-639	750	7

$deposit_2 = \Pi_{account_number,\ balance,\ tuple_id}(employee_info)$

Benefits of Fragmentation:

- **Improved Performance**: Queries can access only the relevant fragments instead of the entire dataset.
- **Parallelism**: Fragments can be processed independently on different nodes.
- **Localization**: Fragments can be stored close to where they are most frequently accessed.

E. Data Transparency

- **Data transparency:** Degree to which system user may remain unaware of the details of how and where the data items are stored in a distributed system
- From the user's perspective, the distributed database should appear as a single, unified system.
- Types of transparency include:
  - ✓ **Location Transparency:** Users don’t need to know where data is located.
  - ✓ **Replication Transparency:** Users don’t need to know about data replication.
  - ✓ **Fragmentation Transparency:** Users don’t need to know how data is partitioned.

Distributed database design

- Refers to the process of designing a database system where data is distributed across multiple physical locations (nodes) connected via a network.
- The **primary goal** is to ensure that the database system is efficient, scalable, reliable, and capable of providing high performance while addressing challenges like data distribution, replication, and consistency.
- Distributed database design involves strategically planning how data is stored and accessed across multiple physical locations (nodes).

Key Considerations

1. Data Distribution

- ❖Ensure that data is appropriately distributed across multiple nodes to balance workload and minimize communication costs. Fragmentation, Replication

2. Transparency

- ❖Provide a user experience where the **distributed nature of the database is hidden**:
  - ✓**Location Transparency**: Users don't need to know where the data resides.
  - ✓**Replication Transparency**: Users don't need to know if data is replicated across sites.
  - ✓**Fragmentation Transparency**: Users don't need to know if data is divided into fragments.

3. Data Locality

- ❖Placing data closer to the users or applications that frequently access it to minimize network latency.

Key Considerations

4. Transaction Management: Ensuring data consistency and preventing conflicts when multiple transactions access and modify data concurrently.

5. Fault Tolerance: Implementing mechanisms to handle node failures or network disruptions, such as data replication and automatic failover.

6. Security: Implementing robust security measures to protect data from unauthorized access and ensure data privacy.

Distributed Database Design Goal

1. Scalability: The system should be able to easily scale to accommodate increasing data volumes and user demands.

2. Reliability and Availability: Ensure the system can **withstand node** or **network failures** while providing uninterrupted service.

3. Performance Optimize query execution and minimize data transfer between nodes.

4. Consistency/Data integrity: Maintain data consistency across nodes, particularly in systems with data replication.

5. Flexibility: The system should be adaptable to changing business requirements.

Steps involved in DDB design

1 Requirement Analysis

- ❖ Understand the application requirements, such as:
  - Data access patterns (e.g., which data is accessed most frequently and by whom).
  - Query types and expected workload (e.g., read-heavy or write-heavy).
  - Performance requirements (e.g., response time, throughput).
  - Reliability and fault tolerance needs.
  - Scalability requirements for future growth.
- ❖ Identify the geographical locations of users and data sources to minimize latency.

Steps involved in DDB design

2. Data Modeling

- ❖Create a high-level logical schema of the database using techniques like Entity-Relationship (ER) modeling to represent data structure.
- ❖Ensure the schema is normalized to remove redundancies and dependencies.
- ❖Define relationships between entities, constraints, and business rules.

Steps involved in DDB design

3. Data Distribution Design:

- **Data Fragmentation:** Fragment data into smaller, manageable pieces based on access patterns and performance requirements.
- **Replicate data:** to improve availability and fault tolerance.

Steps involved in DDB design

4. Data Allocation

- ❖ Decide where to place the fragments across the nodes in the distributed system.
- ❖ Place data on nodes based on access frequency and network topology.
- ❖ Consider the following **data allocation strategies**:
  - **Centralized Allocation**: All fragments/data are stored in a single node (not truly distributed). Simple but lacks scalability and fault tolerance.
  - **Partitioned Allocation**: Each fragment is stored on a single/different node. Reduces storage overhead but may lead to high communication costs.
  - **Replicated Allocation**: Fragments are replicated across multiple nodes for fault tolerance and query performance. But increases storage requirements and consistency management overhead.
  - **Hybrid Allocation**: Some fragments are replicated, and others are partitioned based on access patterns and application requirements.

Steps involved in DDB design

5. Transaction Processing Design:

- Implement concurrency control and failure recovery mechanisms.

6. Performance Evaluation:

- Test the distributed database system to ensure it meets performance requirements.

7. Maintenance and Evolution:

- Continuously monitor and adjust the system as needed.

Key Challenges in Distributed Database Design

- **Data Distribution Complexity:** Determining the optimal way to fragment, allocate, and replicate data is challenging.
- **Consistency Management**
- **Fault Tolerance:** Handling node and network failures while ensuring data integrity and availability.
- **Query Optimization:** Optimizing queries over distributed data to minimize communication and processing costs.
- **Concurrency Control:** Managing concurrent access to distributed data to avoid conflicts and ensure correctness.

Transaction Management in Distributed Databases

❖A **transaction** is a program including a collection of database operations, executed as a logical unit of data processing. The operations performed in a transaction include one or more of database operations like insert, delete, update or retrieve data.

- ✓**read_item()** – reads data item from storage to main memory.
- ✓**modify_item()** – change value of item in the main memory.
- ✓**write_item()** – write the modified value from main memory to storage.

Cont'd

- A transaction that spans across multiple database nodes (may access data at several sites).
- Transactions may comply with the **ACID properties** even when the participating databases are spread over a network.
- Each site has a local **transaction manager** responsible for:
  - ✓ Maintaining a log for recovery purposes
  - ✓ Participating in coordinating the concurrent execution of the transactions executing at that site.
- Each site has a **transaction coordinator**, which is responsible for:
  - ✓ Starting the execution of transactions that originate at the site.
  - ✓ Distributing subtransactions at appropriate sites for execution.
  - ✓ Coordinating the termination of each transaction that originates at the site, which may result in the transaction being committed at all sites or aborted at all sites.

Cont’d

- Each sub-transaction is executed on a different database, but all sub-transactions are part of the same logical transaction.
- Transaction management in distributed databases is a complex but essential aspect of ensuring data integrity and consistency.
- By understanding the challenges and employing appropriate techniques, developers can build reliable and scalable distributed applications.

Challenges in Distributed Transaction Management:

- **Atomicity:** Ensuring that all operations within a transaction are either fully committed or completely rolled back, even if some nodes fail.
- **Consistency:** Maintaining the validity of data across all nodes after a transaction is completed.
- **Isolation:** Preventing interference between concurrent transactions, ensuring that each transaction sees a consistent view of the data.
- **Durability:** Guaranteeing that once a transaction is committed, it will not be lost due to failures.
- **Concurrency Control:** Coordinating access to shared data among multiple transactions to prevent conflicts.

Key Techniques for Distributed Transaction Management:

Commit Protocols

❖Commit protocols are used to ensure atomicity across sites

- ✓ a transaction which executes at multiple sites must either be committed at all the sites, or aborted at all the sites.
- ✓not acceptable to have a transaction committed at one site and aborted at another.

❖The two-phase commit (2PC) protocol is widely used

❖The three-phase commit (3PC) protocol is more complicated and more expensive, but avoids some drawbacks of two-phase commit protocol. This protocol is not used in practice.

Key Techniques for Distributed Transaction Management:

1. Two-Phase Commit (2PC):

❖**A standard protocol for ensuring atomicity.**

❖**Involves two phases:**

- **Voting Phase:** The coordinator asks all participants if they can commit.
- **Commit/Rollback Phase:** If all participants vote yes, the coordinator instructs them to commit; otherwise, it instructs them to rollback.

❖**Ensures atomicity but can be susceptible to blocking in case of failures.**

Cont’d

2. Three-Phase Commit (3PC):

- An extension of 2PC that addresses some of its limitations.
- Introduces an intermediate precommit phase to reduce the risk of blocking.
- More complex but can improve availability in certain failure scenarios.

Cont’d

3. Concurrency Control Mechanisms:

- **Locking:** A common technique where nodes acquire locks on data before accessing it, preventing other transactions from modifying the same data concurrently.
- **Timestamp Ordering:** Assigns timestamps to transactions and ensures that operations are executed in the order of their timestamps.
- **Optimistic Concurrency Control:** Assumes that conflicts are rare and only checks for conflicts at the end of a transaction. If conflicts occur, the transaction is aborted and retried.

Cont’d

4. Distributed Deadlock Detection:

- Algorithms to detect and resolve deadlocks, where two or more transactions are waiting for each other to release resources.
- Common approaches include centralized and distributed deadlock detection algorithms.

Failures in a Distributed System

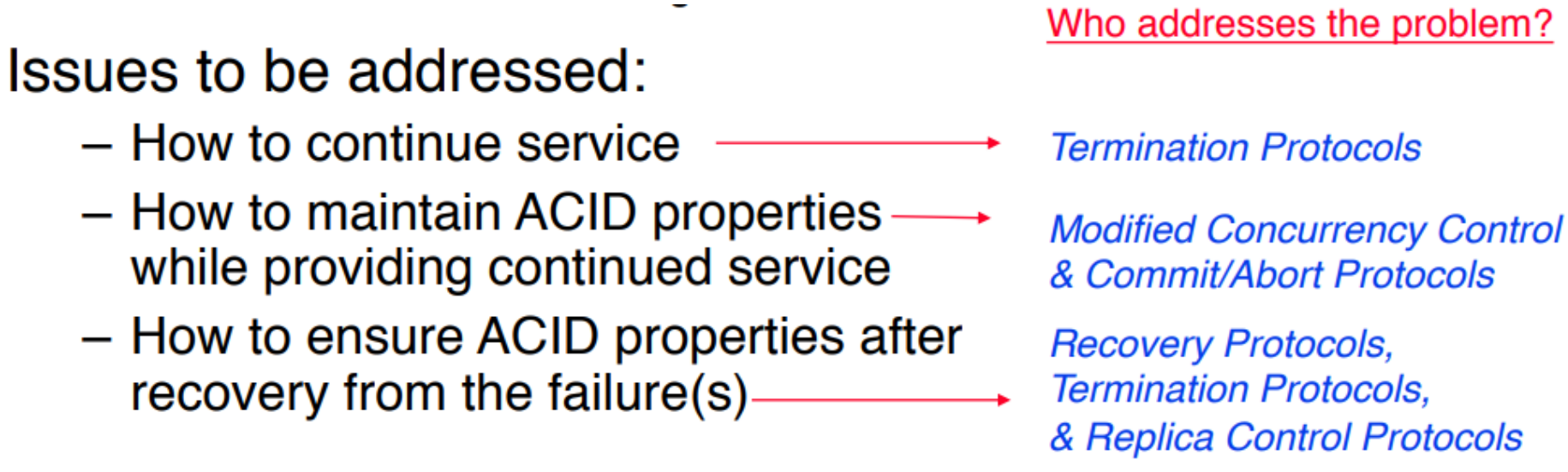
❖**Types of Failure:**

– Transaction failure

– Node failure

– Media failure

– Network failure



Distributed Query Processing

- Distributed Query Processing refers to the process of executing a query in a distributed database system where data is stored/spread across multiple sites or nodes, connected via a network.
- The goal of distributed query processing is to retrieve results efficiently while minimizing costs such as communication overhead, data transfer, response time and maximize performance by strategically distributing the processing workload among the available nodes.
- For centralized systems, the primary criterion for measuring the cost of a particular strategy is the number of **disk accesses**.
- In a distributed system, other issues must be taken into account, the **cost of a data transmission over the network** and other.

Key Components of Distributed Query Processing

A. **Query Decomposition:** The input query (usually in SQL) is decomposed into smaller subqueries or operations, which can be executed independently or in parallel on different nodes.

B. **Data Localization:** Determines where the data required by the query resides. Subqueries are directed to the appropriate nodes storing the relevant data.

C. **Query Optimization:** Focuses on reducing the cost of query execution. Choosing the most efficient execution plan involves considering various factors, such as data locality, amount of data transfer, communication costs, and available processing power at each node.

D. **Query Execution:** Executes subqueries on the distributed nodes. Collects and aggregates the results from all the nodes to form the final result.

Query Optimization Techniques/Common Approaches

1. Join Optimization: Minimize the cost of joining tables across nodes by:

- Reducing data transfer.
- Using semi-joins to prune unnecessary data.

2. Data Reduction/ Heuristic Optimization Apply filters/selection and projections at the local nodes where the data resides to reduce the amount of data sent over the network to optimize query execution.

3. Parallelism: Leverage parallel processing by executing subqueries simultaneously on multiple nodes.

Query Optimization Techniques/Common Approaches

❖**Data Shipping:** Moving data to the node where the query originates or to a central processing node. This is suitable when the amount of data to be transferred is relatively small.

❖Ship copies of all three relations to site S_I and choose a strategy for processing the entire locally at site S_I.

❖Ship a copy of the **account** relation to site **S2** and compute **temp1** = **account** ⋈ **depositor** at **S2**. Ship **temp1** from **S2** to **S3**, and compute **temp2** = **temp1** ⋈ **branch** at **S3**. Ship the result **temp2** to **SI**.

❖Must consider following factors:

- amount of data being shipped
- cost of transmitting a data block between sites
- relative processing speed at each site

Example of Distributed Query Processing

- **Example of Distributed Query Processing**
- Consider a distributed database with two tables:
- **Table A** is stored in Node 1.
- **Table B** is stored in Node 2.
- **Query**:

```
SELECT A.name, B.salary
FROM A, B
WHERE A.id = B.id AND B.salary > 50000;
```

Example of Distributed Query Processing

1.Decomposition:

Break the query into subqueries:

- Retrieve rows from B where salary > 50000 (processed locally on Node 2).
- Perform a join between A and the filtered rows of B.

2.Optimization:

- Push the salary > 50000 condition to Node 2 to reduce transferred data.
- Use a semi-join to minimize data movement during the join operation.

3.Execution:

- Node 2 sends only the filtered rows of B to Node 1.
- Node 1 performs the join and returns the final result.